实验三 芯片的封装及应用

一、实验目的

1. 掌握 Verilog 语言框架，编程及调试的方法；
2. 掌握 74LS161 芯片的功能、实现及应用，掌握竞争冒险的原因及解决方法；
3. 掌握 IP 核封装、原理图/块设计方法及 FPGA 开发板的使用。

二、实验内容

1. 完成 74LS161 计数器芯片的实现、测试及 6 进制计数器的实现，将程序下载到 FPGA 开发板进行验证；
2. 分析电路中的竞争与冒险，给出解决方案并验证；
3. 自主设计方法解决竞争冒险（选做）；
4. 将芯片及相关模块封装为 IP 核，通过原理图/块设计实现 N 进制计数器，观察、分析仿真波形和电路图；
5. 大于 16 的计数器实现（选做）；
6. 倒计时的实现（选做）。

三、实验要求

1.说明电路功能，分析设计、仿真代码和电路图。
2.分析仿真波形，观察输入输出是否与预期电路功能相符（测试要全面，关注特殊情况的测试）。
3.记录设计和调试过程。

四、实验过程及结果分析

对电路功能、设计代码、电路图、测试结果等进行简要分析说明，不能只贴截图。

4.1 74LS161 芯片设计与应用

一、74LS161：
1、设计文件：

```
module Counter_74LS161 #(parameter WIDTH = 4)(
    input CR_n,                    //异步清零
```

```
    input CP,                       //时钟
    input [WIDTH - 1 : 0] D,        //数据输入
    input LD_n,                     //置数控制
    input EP, ET,                   //计数使能
    output [WIDTH - 1 : 0] Q,       //计数输出
    output RCO                      //进位输出
);
    reg [WIDTH - 1 : 0] Data_out;
    assign RCO = (Data_out == {WIDTH{1'b1}}) && ET;
    always @(posedge CP or negedge CR_n) begin
        if (!CR_n) Data_out <= {WIDTH{1'b0}};
        else if (!LD_n) Data_out <= D;
        else if (EP && ET) Data_out <= Data_out + 1;
    end
    assign Q = Data_out;
endmodule
```

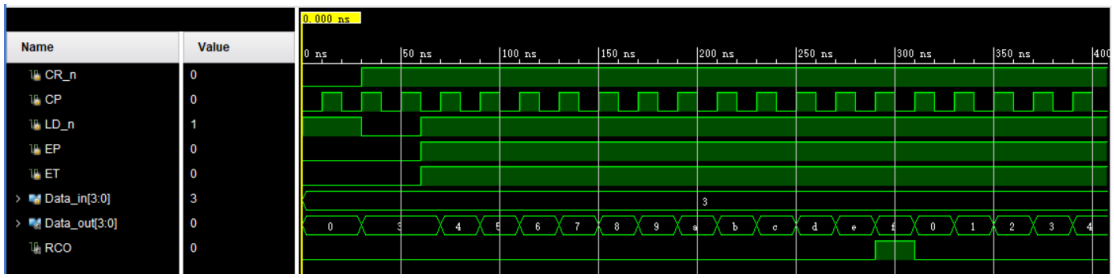
实现 4 位同步二进制计数器 74LS161 的核心逻辑，支持异步清零、同步置数、计数使能和进位输出。当输出为 1111 且高使能时，产生进位；在时钟上升沿或清零信号下降沿时，若清零信号生效，则输出为 0；若置数信号（load）有效，则输出保持不变；若这两者都无效，且使能有效，则正常计数，输出+1。

2、仿真文件：

```
module sim_74LS161;
    reg CR_n, CP, LD_n, EP, ET;
    reg [3:0] Data_in;
    wire [3:0] Data_out;
    initial begin
        EP = 0; ET = 0; CR_n = 0; LD_n = 1; CP = 0; Data_in = 4'b0011;
        #30 CR_n = 1; LD_n = 0;
        #30 LD_n = 1; EP = 1; ET = 1;
        #800 CR_n = 0;
    end
    always #10 CP = ~CP;
    Counter_74LS161 T(CR_n, CP, Data_in, LD_n, EP, ET, Data_out, RCO);
endmodule
```

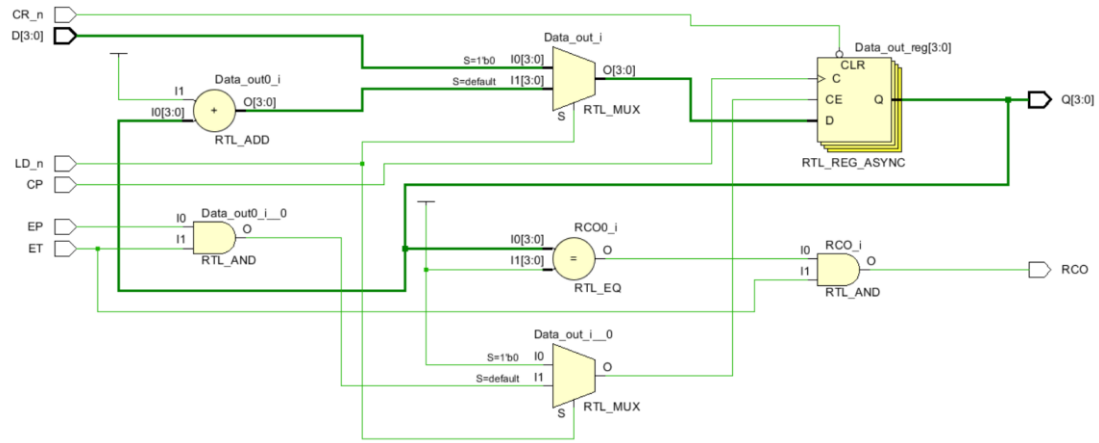
通过设置清零信号、置数信号与使能信号，并加以规律的时钟，可以观察到不同情况下计数器的表现。

3、SIMULATIOIN：



0-30ns，清零信号有效，输出始终为 0；30-60ns，清零信号无效，置数信号有效，输出始终保持与输入 0011 一致；60ns 后，这两个信号都无效，使能信号有效，74LS161 开始正常计数，在计到 15（1111）后产生进位 RCO 并将输出置 0。可以看到，仿真结果符合电路全部功能的预期。

4、RTL ANALYSIS：



二、六进制计数器：

1、设计文件：

(1)时钟:

```
module CP_M(output reg CP);
    initial CP = 1'b1;
    always #10 CP = ~CP;
endmodule
```

初始为高电平，每隔 10ns 翻转一次。

(2)二输入与非门:

```
module nand2(
    input a, b,
    output y
);
    assign y = ~(a & b);
endmodule
```

将输入信号进行与非门运算后输出。

(3)六进制计数器：

```
module Counter_74LS161_6(
    input LD_n, EP, ET,
    output wire [3:0] Data_out
);
    CP_M C(CP);
    Counter_74LS161 T(CR_n, CP, 4'b0000, LD_n, EP, ET, Data_out, RCO);
```

```
    nand2 N(Data_out[2], Data_out[1], CR_n);
endmodule
```

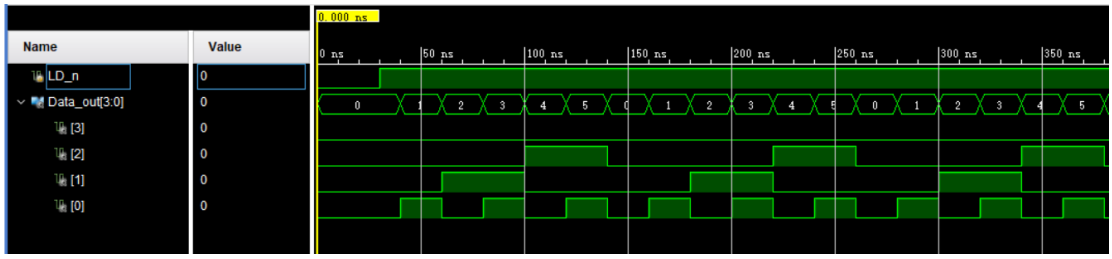
通过 CP_M 构建时钟 CP，给 74LS161 传入初始的清零信号、时钟信号、使能信号与起始输入 0000，输出 Data_out 和 RCO。最后通过与非门，识别 0110（6），在计到 6 时激活清零信号让计数器复位，实现六进制计数。

2、仿真文件：

```
module sim_74LS161_6;
  reg LD_n;
  wire [3:0] Data_out;
  initial begin
    LD_n = 0;
    #30 LD_n = 1;
  end
  Counter_74LS161_6 T(LD_n, 1'b1, 1'b1, Data_out);
endmodule
```

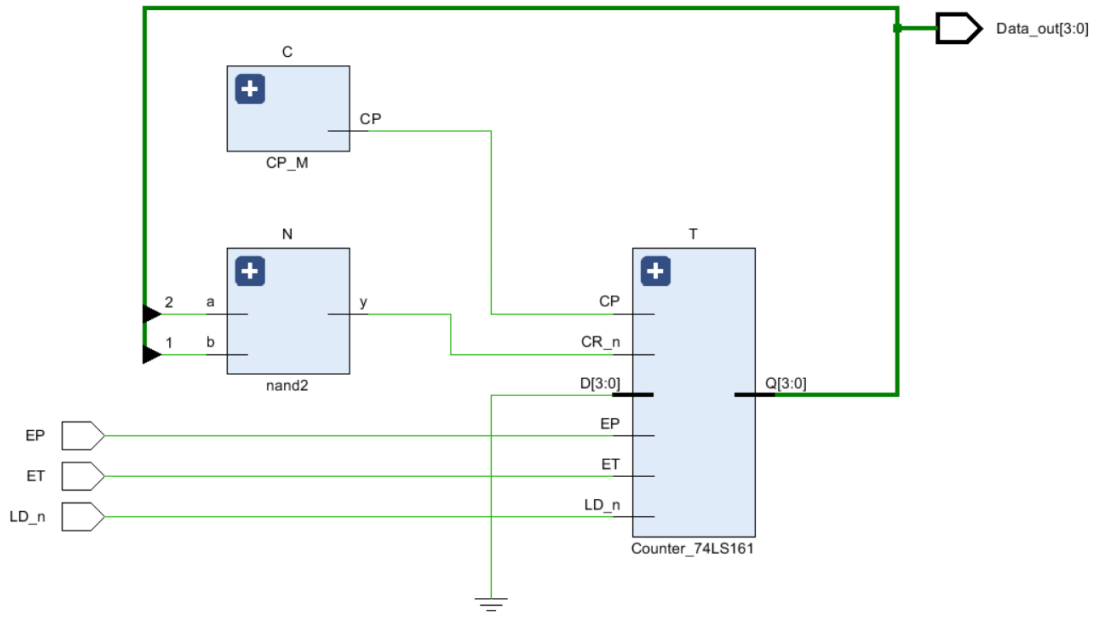
初始时置数信号有效，30ns 后无效。将置数信号和高使能信号传入六进制计数器。

3、SIMULATION：



置数信号有效时输出始终为 0，无效后开始正常计数，到 5 后计够 6 个数，返回 0，实现六进制计数功能，符合预期。

4、RTL ANALYSIS：



5、开发板实现：

（1）时钟分频模块：

```
module clk_div(
    input clk_100M, //输入时钟信号
    output clk_100, clk_50, clk_20, clk_10, clk_5, clk_1);  //输出时钟信号
    reg [15:0] cnt_khz;
    reg [11:0] cnt_hz;
    reg clk_1k, clk_100_reg, clk_50_reg, clk_20_reg, clk_10_reg, clk_5_reg,
clk_1_reg ;
    initial begin
        cnt_khz = 0; cnt_hz = 0; clk_1k = 0; clk_100_reg = 0; clk_50_reg = 0;
clk_20_reg = 0; clk_10_reg = 0; clk_5_reg = 0;clk_1_reg = 0;
    end
    always @(posedge clk_100M) begin
        cnt_khz <= cnt_khz + 1;
        if(cnt_khz == 16'hC34F)
            clk_1k <= ~clk_1k;
    end
    always @(posedge clk_1k) begin
        cnt_hz <= cnt_hz + 1;
    end
    always @(posedge clk_1k) begin   //通过计数实现分频
        if(cnt_hz[2:0] == 3'b100) clk_100_reg <= ~clk_100_reg;
        else if(cnt_hz[3:0] == 4'b1001) clk_50_reg <= ~clk_50_reg;
        else if(cnt_hz[4:0] == 5'b11000) clk_20_reg <= ~clk_20_reg;
        else if(cnt_hz[5:0] == 6'b110001) clk_10_reg <= ~clk_10_reg;
        else if(cnt_hz[6:0] == 7'b110011) clk_5_reg <= ~clk_5_reg;
        else if(cnt_hz[8:0] == 9'b11_1110_011) clk_1_reg <= ~clk_1_reg;
    end
    assign clk_100 = clk_100_reg;
    assign clk_50 = clk_50_reg;
    assign clk_20 = clk_20_reg;
    assign clk_10 = clk_10_reg;
    assign clk_5 = clk_5_reg;
    assign clk_1 = clk_1_reg;
endmodule
```

（2）数码管模块：

```
module hex7seg(
    input wire x0, x1, x2, x3,
    output wire [3:0] an,
    output reg [6:0] seg
);
    wire [3:0] S;
```

```
    assign an = 4'b1110;
    assign S = {x3, x2, x1, x0};
    always @(*) case(S)
        0 : seg = 7'b1000000;
        1 : seg = 7'b1111001;
        2 : seg = 7'b0100100;
        3 : seg = 7'b0110000;
        4 : seg = 7'b0011001;
        5 : seg = 7'b0010010;
        6 : seg = 7'b0000010;
        7 : seg = 7'b1111000;
        8 : seg = 7'b0000000;
        9 : seg = 7'b0010000;
        'hA : seg = 7'b0001000;
        'hB : seg = 7'b0000011;
        'hC : seg = 7'b1000110;
        'hD : seg = 7'b0100001;
        'hE : seg = 7'b0000110;
        'hF : seg = 7'b0001110;
        default : seg = 7'b0000001;
    endcase
endmodule
```

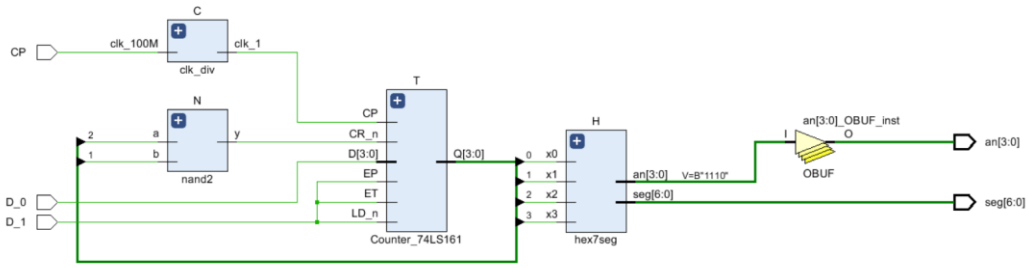
将 x0,x1,x2,x3 组合成 S，设置每个十六进制数字在数码管上的显示。

（3）Basys3 实现六进制计数器：

```
module Counter_74LS161_6_Basys3(
    input CP, D_0, D_1,
    output wire [3:0] an, wire [6:0] seg
);
    wire clk_1;
    clk_div C (.clk_100M(CP), .clk_1(clk_1));
    Counter_74LS161 T(CR_n, clk_1, {D_0, D_0, D_0, D_0}, D_1, D_1, D_1, {Q3, Q2,
Q1, Q0}, RCO);
    hex7seg H(Q0, Q1, Q2, Q3, an, seg);
    nand2 N(Q2, Q1, CR_n);
endmodule
```

设置好时钟，输入，数码管，重置条件等，以便在开发板上观察结果，原理同 Counter_74LS161_6。

（4）RTL ANALYSIS：



（5）约束文件：

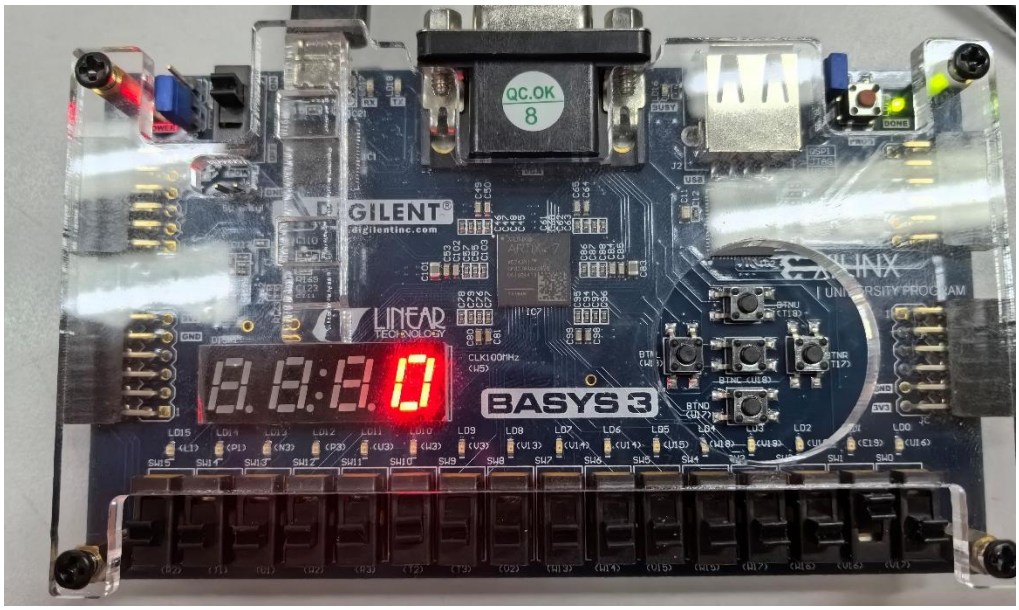
```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports CP]
    set_property IOSTANDARD LVCMOS33 [get_ports CP]

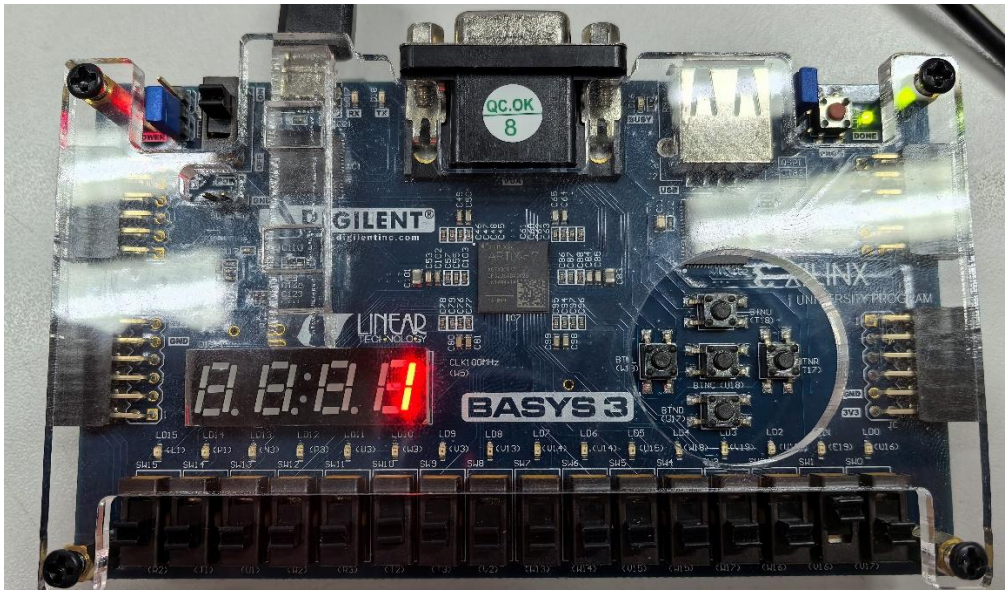
## Switches
set_property PACKAGE_PIN V17 [get_ports D_0]
    set_property IOSTANDARD LVCMOS33 [get_ports D_0]
set_property PACKAGE_PIN V16 [get_ports D_1]
    set_property IOSTANDARD LVCMOS33 [get_ports D_1]

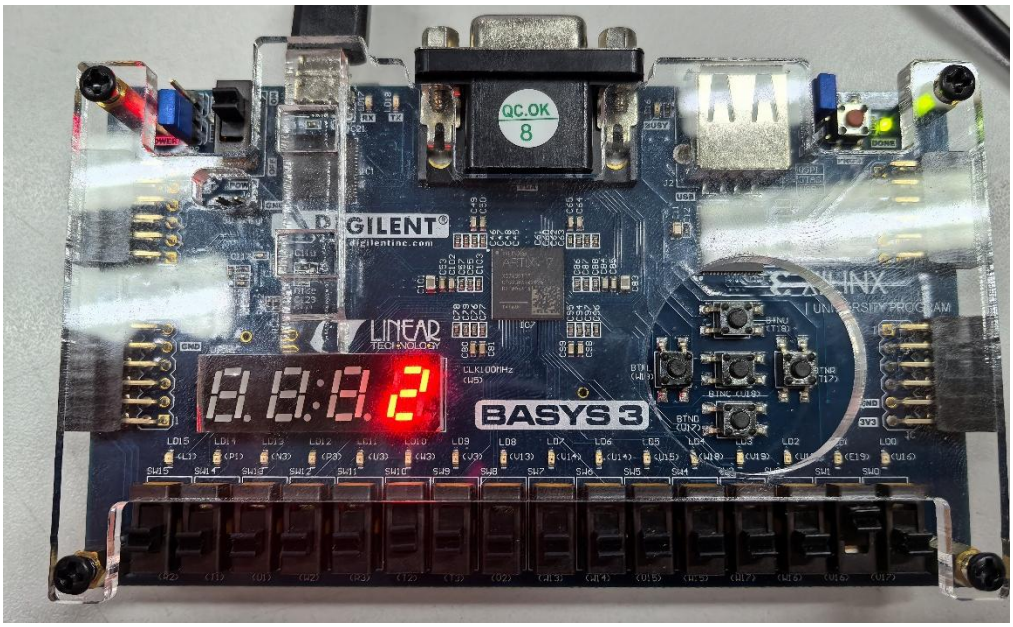
##7 segment display
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

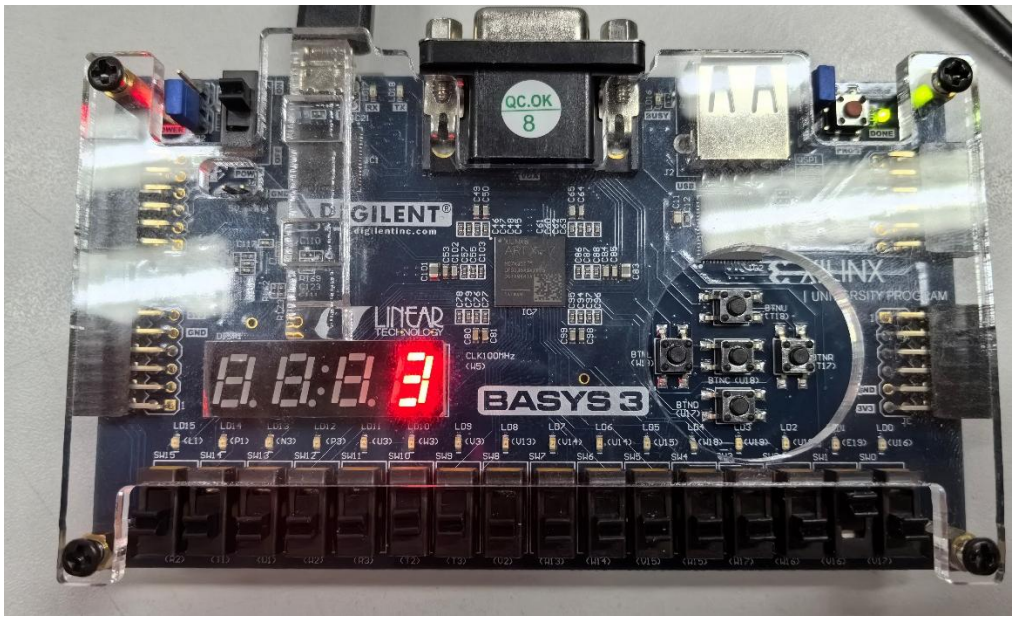
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

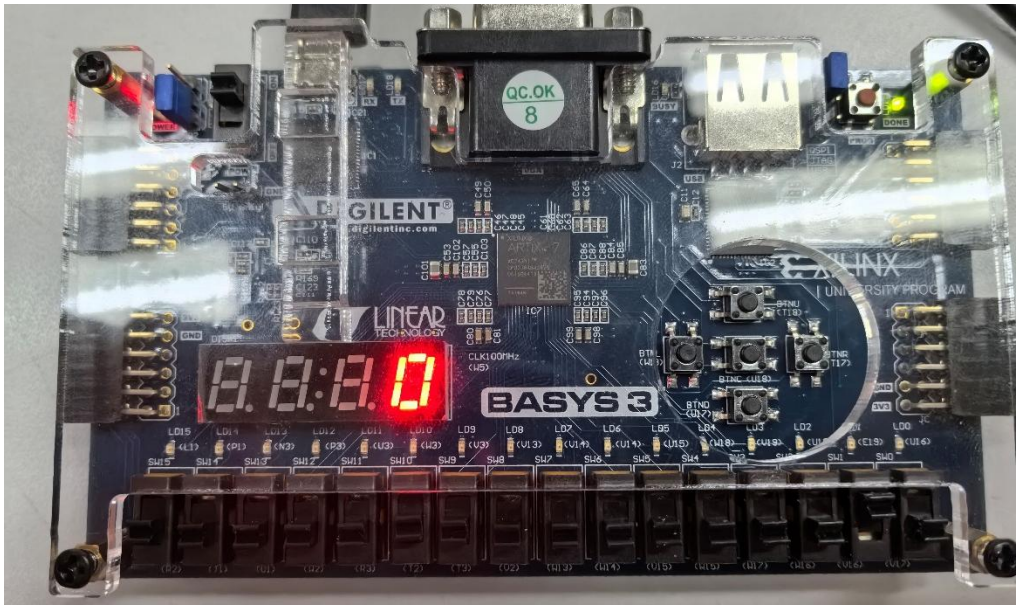
（6）烧录结果：


QC.OK
8
GILENT
digilentinc.com
LINEAR
TECHNOLOGY
BASYS 3


QC.OK
8
GILENT
digilentinc.com
LINEAR
TECHNOLOGY
BASYS 3


QC.OK
8
GILENT
digilentinc.com
LINEAR
TECHNOLOGY
BASYS 3





计数 0，1，2，3 后便重置为 0，实现的是四进制计数器的功能，是因为过程中发生了竞争冒险现象。

4.2 竞争冒险的分析与解决

一、对竞争冒险现象的分析：

1、概念：

不同路径输入信号变化传输到同一点门级电路，时间上有先后，时间差称为竞争。

因竞争导致在输出端可能产生尖峰脉冲的现象，称为冒险。

竞争表现在输出波形上，则是出现 0 电平或者 1 电平的尖峰，称 “毛刺”。

2、分析：

六进制计数器在烧录后可能变成四进制计数器的原因：

在计到3时，Q2Q1=01；此时要变成4，则这两位都需要翻转，发生竞争现象。若先变为Q2Q1=00，

再变为 10，则正常计数；若先变为 Q2Q1=11，则直接出现 6（110），符合重置条件，产生进位并置 0，从而变成四进制计数器。

二、竞争的消除方法：

1、使用时钟同步；
2、修改逻辑增加冗余项；
3、用格雷码计数器；
4、增加滤波电容，滤除窄脉冲。

三、修改代码实现竞争的消除：

1、时钟同步：

```
    always @(posedge CP) begin
        if (!CR_n) Data_out <= {WIDTH{1'b0}};          //同步清零
        else if (!LD_n) Data_out <= D;                 //同步置数
        else if (EP && ET) Data_out <= Data_out + 1;
    end
```

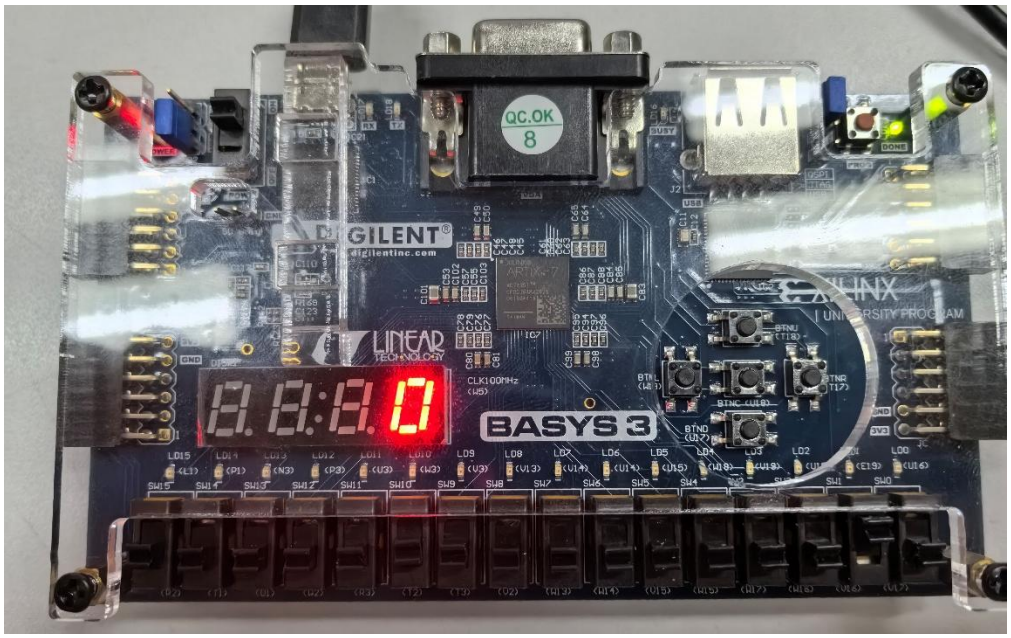
采用同步清零，避免竞争与毛刺。

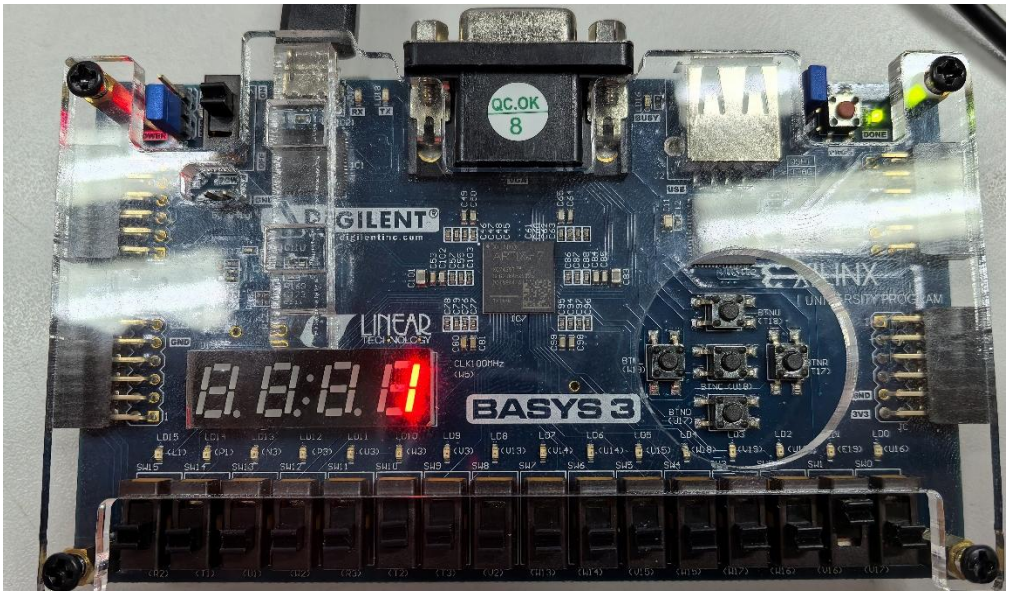
2、修改与非门：

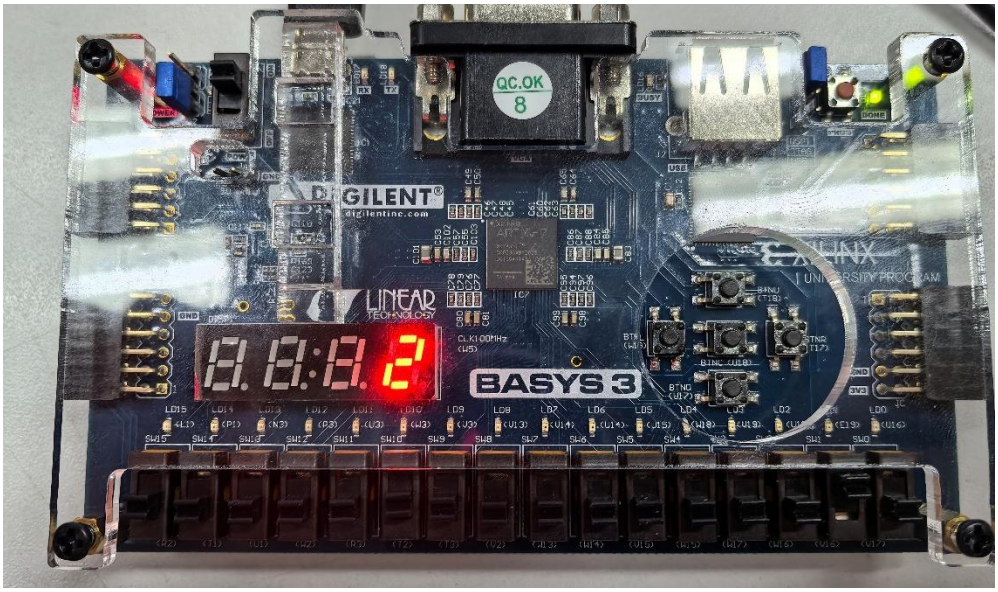
```
module nand2_CP(
    input a, b,
    output y,
    input CP
);
    reg yi, yt;
    assign y = yi;
    always @(negedge CP) begin
        yt <= ~(a & b);
    end
    always @(posedge CP) begin
        yi <= yt;
    end
endmodule
```

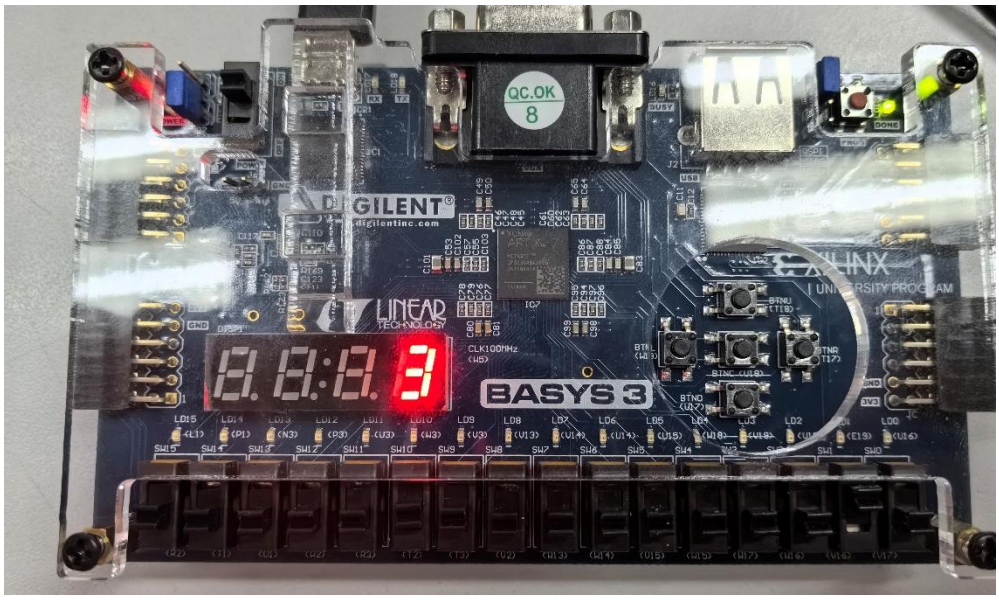
与非门由时钟下降沿统一控制，避免发生竞争。

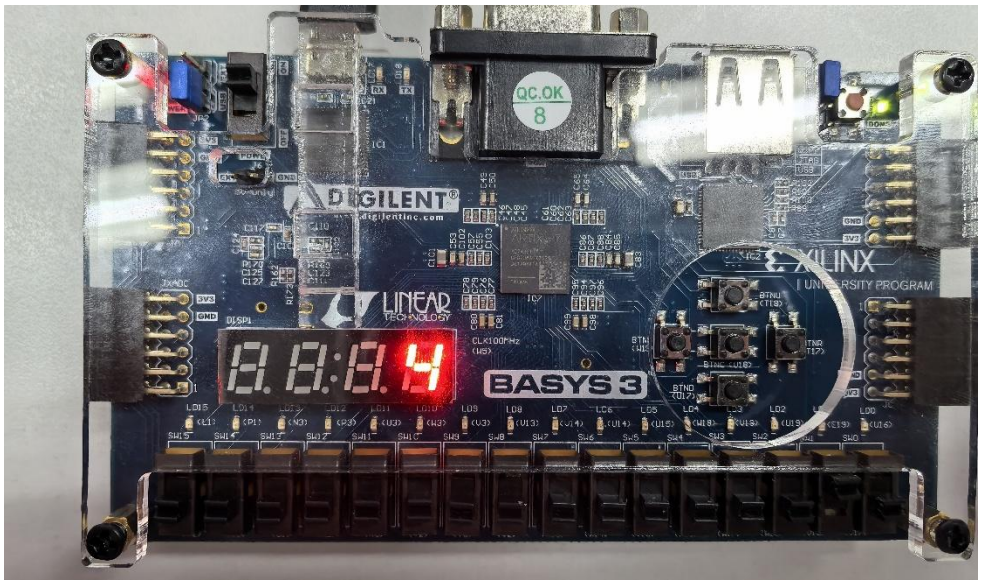
3、修改后烧录结果：

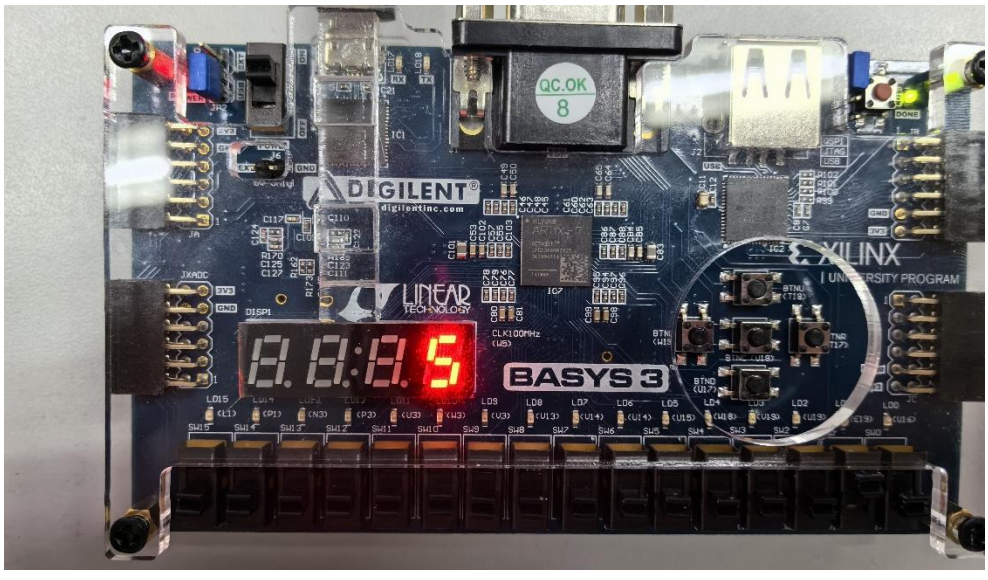

QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM


QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM


QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM


QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
XILINX
BASYS 3


QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
XILINX
BASYS 3


QC.OK
8
DIGILENT
digilentinc.com
LINEAR TECHNOLOGY
XILINX
BASYS 3

计数器可以正常从 0 计到 5，说明竞争冒险现象已被避免，实现了预期中的功能。

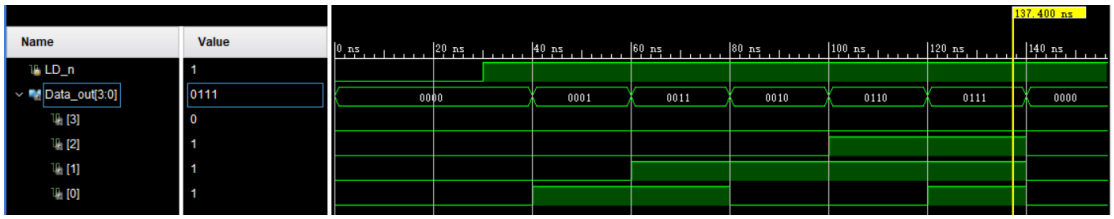
4.3 自主设计方法解决竞争冒险（选做）

1、用典型 Gray 码进行计数：

```
module Counter_74LS161_6(
    input LD_n, EP, ET,
    output wire [3:0] Data_out
);
    wire CP, CR_n;
    wire [3:0] Q;
    wire RCO;
    wire [3:0] gray_count;
    assign gray_count = {Q[3], Q[3:1] ^ Q[2:0]};
    assign Data_out = gray_count;
    CP_M C(CP);
    Counter_74LS161
T(.CR_n(CR_n), .CP(CP), .D(4'b0000), .LD_n(LD_n), .EP(EP), .ET(ET), .Q(Q), .RCO
(RCO));
    wire clear_cond = (gray_count == 4'b0101);
    nand2 N(clear_cond, 1'b1, CR_n);
endmodule
```

通过逻辑构造格雷码，在格雷码位 0101 时启动清零信号，实现六进制计数。因为格雷码每次只变化一位，因此可以有效地避免竞争冒险。

2、SIMULATION：



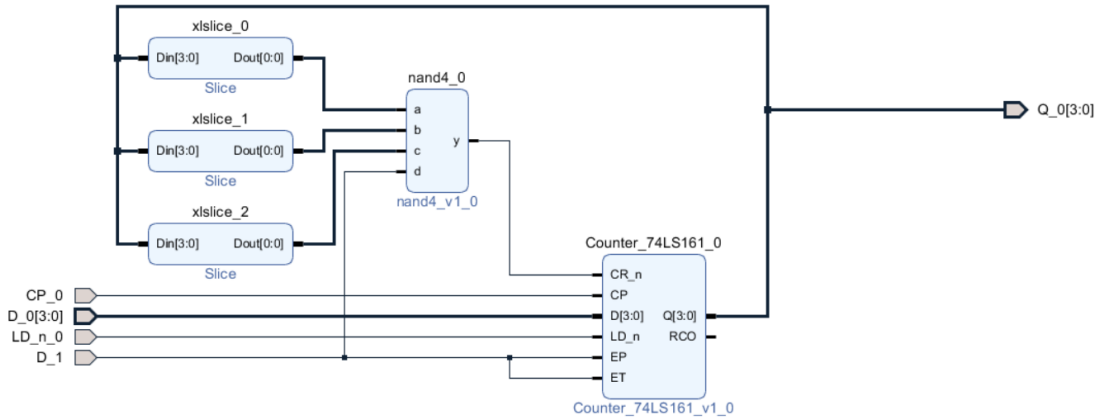
由仿真波形可见，实现了格雷码计数的功能。

4.4 原理图/块设计实现 N 进制计数器

（N = 学号最后一位数字 + 5）

N = 13（8+5）

1、 原理图：



要实现 13 进制（1101）计数，应该将 xlslice 设置第 0、2、3 位接入四输出与非门，用于判断是否启动清零信号。当计到 13 时，启动同步清零信号将计数置 0。

2、 代码：

（1）原型：

```
`timescale 1 ps / 1 ps

(*                                CORE_GENERATION_INFO                                =
"design_1,IP_Integrator,{x_ipVendor=xilinx.com,x_ipLibrary=BlockDiagram,x_ipNam
e=design_1,x_ipVersion=1.00.a,x_ipLanguage=VERILOG,numBlks=5,numReposBlks=5,num
NonXlnxBlks=0,numHierBlks=0,maxHierDepth=0,numSysgenBlks=0,numHlsBlks=0,numHdlr
efBlks=0,numPkgbdBlks=0,bdsource=USER,synth_mode=OOC_per_IP}" *) (* HW_HANDOFF =
"design_1.hwdef" *)
module design_1
   (CP_0,
    D_0,
    D_1,
    LD_n_0,
    Q_0);
  input CP_0;
  input [3:0]D_0;
  input D_1;
  input LD_n_0;
  output [3:0]Q_0;

  wire CP_0_1;
```

```
  wire [3:0]Counter_74LS161_0_Q;
  wire [3:0]D_0_1;
  wire EP_0_1;
  wire LD_n_0_1;
  wire nand4_0_y;
  wire [0:0]xlslice_0_Dout;
  wire [0:0]xlslice_1_Dout;
  wire [0:0]xlslice_2_Dout;

  assign CP_0_1 = CP_0;
  assign D_0_1 = D_0[3:0];
  assign EP_0_1 = D_1;
  assign LD_n_0_1 = LD_n_0;
  assign Q_0[3:0] = Counter_74LS161_0_Q;
  design_1_Counter_74LS161_0_0 Counter_74LS161_0
       (.CP(CP_0_1),
        .CR_n(nand4_0_y),
        .D(D_0_1),
        .EP(EP_0_1),
        .ET(EP_0_1),
        .LD_n(LD_n_0_1),
        .Q(Counter_74LS161_0_Q));
  design_1_nand4_0_0 nand4_0
       (.a(xlslice_0_Dout),
        .b(xlslice_1_Dout),
        .c(xlslice_2_Dout),
        .d(EP_0_1),
        .y(nand4_0_y));
  design_1_xlslice_0_0 xlslice_0
       (.Din(Counter_74LS161_0_Q),
        .Dout(xlslice_0_Dout));
  design_1_xlslice_1_0 xlslice_1
       (.Din(Counter_74LS161_0_Q),
        .Dout(xlslice_1_Dout));
  design_1_xlslice_2_0 xlslice_2
       (.Din(Counter_74LS161_0_Q),
        .Dout(xlslice_2_Dout));
endmodule
```

（2）封装：

```
module design_1_wrapper
   (CP_0,
    D_0,
    D_1,
```

```
    LD_n_0,
    Q_0);
  input CP_0;
  input [3:0]D_0;
  input D_1;
  input LD_n_0;
  output [3:0]Q_0;

  wire CP_0;
  wire [3:0]D_0;
  wire D_1;
  wire LD_n_0;
  wire [3:0]Q_0;

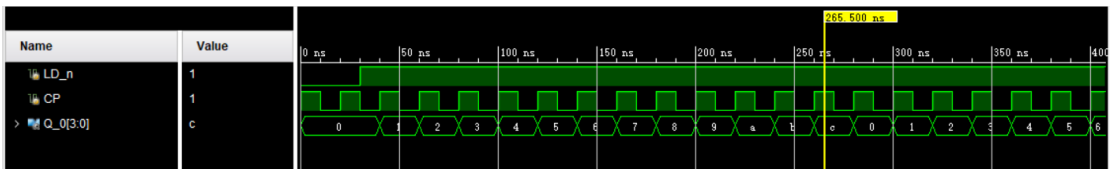
  design_1 design_1_i
       (.CP_0(CP_0),
        .D_0(D_0),
        .D_1(D_1),
        .LD_n_0(LD_n_0),
        .Q_0(Q_0));
endmodule
```

3、仿真文件：

```
module sim;
    reg LD_n = 0, CP = 1;
    wire [3:0] Q_0;
    initial begin
        #30 LD_n = 1;
    end
    always #10 CP = ~CP;
    design_1_wrapper T(CP, 4'b0000, 1'b1, LD_n, Q_0);
endmodule
```

与先前仿真代码类似，30s 后关闭置数信号，开始正常计数。

4、SIMULATION：



在计数 0-c（12）后，数满 13 个数字，计数器归零。由仿真图可见，符合预期。

4.5 大于 16 的计数器实现（选做）

1、24 进制计数器设计代码：

```
module Counter24(
    input LD_n, EP, ET,
    output wire [4:0] Data_out
);
    wire CP;
    CP_M C(CP);

    wire CR_n;
    wire RCO_low;
    wire [3:0] Q_low;
    wire [3:0] Q_high;

    //当计数=23（10111）时清零
    assign CR_n = ~(Q_high[0] & ~Q_low[3] & Q_low[2] & Q_low[1] & Q_low[0]);

    assign Data_out = {Q_high[0], Q_low};

    // 低位计数器
    Counter_74LS161
low_counter( .CR_n(CR_n), .CP(CP), .D(4'b0000), .LD_n(LD_n), .EP(EP), .ET(ET),
.Q(Q_low), .RCO(RCO_low));

    // 高位计数器
    Counter_74LS161
high_counter( .CR_n(CR_n), .CP(CP), .D(4'b0000), .LD_n(LD_n), .EP(RCO_low), .ET
(RCO_low), .Q(Q_high), .RCO());
endmodule
```

用两个 74LS161 级联，一个表示高四位一个表示低四位，在低位计数器产生进位（到达 1111）时启动高位 74LS161，实现大于 16 的计数。在数到 23（10111），即高位 Q0 与低位 Q0Q1Q2 都为 1 时启动清零信号，从而实现 24 进制计数器。

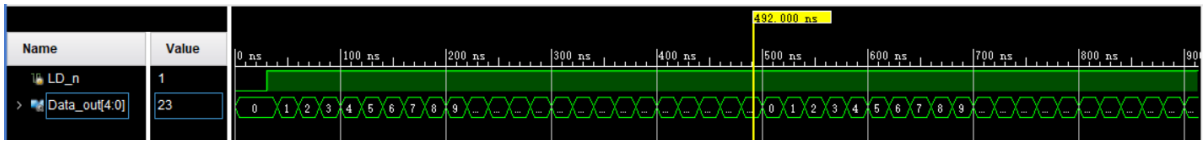
2、仿真代码：

```
module sim_Counter24;
    reg LD_n;               //置数控制
    wire [4:0] Data_out;    //计数输出
    initial begin
        LD_n = 0;           //初始化赋值
        #30 LD_n = 1;       //置数
    end
    //实例化
```

```
    Counter24 T(LD_n, 1'b1, 1'b1, Data_out);
endmodule
```

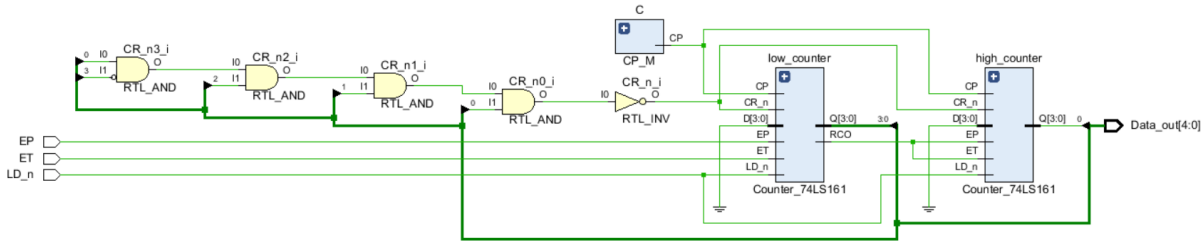
与先前仿真代码类似。

3、SIMULATION：



由仿真波形图可见计数 0-23 后回到 0，实现 24 进制计数器，符合电路预期。

4、RTL ANALYSIS：



由电路图可以看到两个 74LS161 的级联结构。

4.6 倒计时的实现（选做）

1、修改 74LS161，使其进行减法计数：

```
module Counter_74LS161 #(parameter WIDTH = 4)(
    input CR_n,
    input CP,
    input [WIDTH-1:0] D,
    input LD_n,
    input EP, ET,
    output [WIDTH-1:0] Q,
    output RCO
);
    reg [WIDTH-1:0] Data_out;

    assign RCO = (Data_out == {WIDTH{1'b0}}) && ET ? 1'b0 : 1'b1;

    always @(posedge CP) begin
        if (!CR_n)
            Data_out <= {WIDTH{1'b0}};
        else if (!LD_n)
            Data_out <= D;
```

```
        else if (EP && ET)
            Data_out <= Data_out - 1;
    end

    assign Q = Data_out;
endmodule
```

将加法计数改为减法，当出现 0000 时启动进位信号，回到 1111 继续减法。

2、六进制倒计时：

```
module DownCounter6(
    input LD_n,
    input EP,
    input ET,
    output wire [3:0] Data_out
);
    wire CP;
    wire RCO;
    wire [3:0] Q;
    reg power_on_reset;
    CP_M C(CP);

    wire [3:0] D = 4'b0101;

    initial begin
        power_on_reset = 1'b0;
        #10 power_on_reset = 1'b1;
    end

    wire internal_LD_n = RCO && LD_n;

    Counter_74LS161
counter( .CR_n(power_on_reset), .CP(CP), .D(D), .LD_n(internal_LD_n), .EP(EP),
.ET(ET), .Q(Q), .RCO(RCO));
    assign Data_out = Q;
endmodule
```

以 0101 为开始进行倒计时，计到 0000 后返回 0101，继续倒计时，实现六进制倒计时的功能。

2、仿真文件：

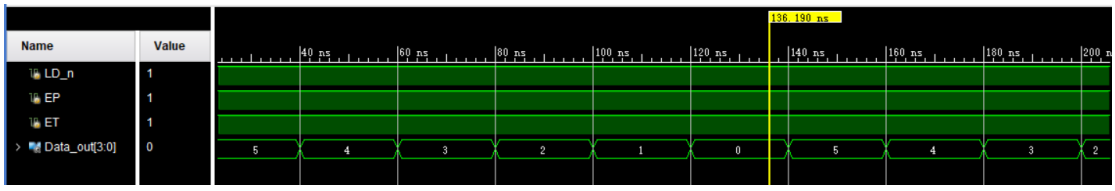
```
module sim_DownCounter;
    reg LD_n;
    reg EP;
    reg ET;
    wire [3:0] Data_out;
```

```
    initial begin
        LD_n = 1;
        EP = 1;
        ET = 1;
    end

    DownCounter6 t(.LD_n(LD_n), .EP(EP), .ET(ET), .Data_out(Data_out));
endmodule
```

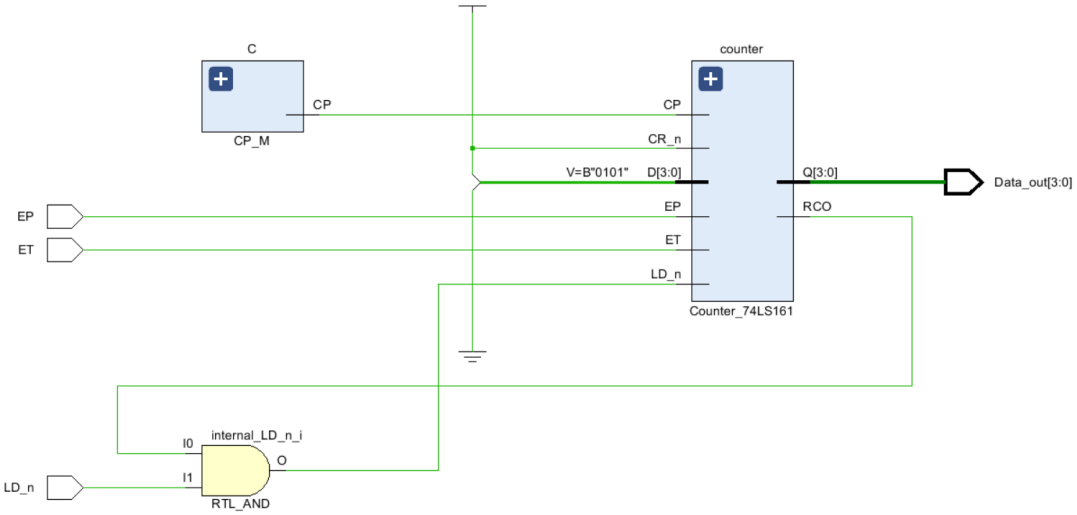
启动使能信号，关闭置数信号，直接观察电路功能。

3、SIMULATION：



由仿真波形图可见，倒计时从 5 数到 0 后又回到 5，实现了预期的六进制倒计时功能。

4、RTL ANALYSIS：



五、调试和心得体会

记录实验过程中遇到的问题和调试解决过程，心得体会等。

1、问题调试：

（1）竞争冒险的调试：

在实现六进制计数器的过程中，发现原始设计烧录到 FPGA 开发板后出现计数异常现象，本应实现六进制计数功能，但实际表现为四进制计数。经过详细的波形仿真与逻辑分析，发现问题出在异步清零信号与计数信号竞争冒险导致的尖峰脉冲（毛刺）。具体来说，当计数器从状态 011（3）向 100（4）转换时，由于多个输出位同时翻转，导致短暂状态跳变至 110

（6），激活了提前清零逻辑，进而形成竞争冒险。

为解决这一问题，通过以下方法进行调试和优化：

- 采用时钟同步方法，将原有异步清零改为时钟上升沿同步清零，避免了异步信号的不确定性。
- 设计了与非门的同步版本，通过时钟边沿锁存信号，进一步消除了可能存在的毛刺。
- 采用 Gray 码进行计数以避免竞争冒险。

通过仿真波形验证与实际烧录，验证这些方法有效避免了计数状态的同时多位跳变，竞争冒险现象得到了彻底的消除。经过仿真和 FPGA 板上的实际运行，修改后的六进制计数器的运行稳定，计数准确。

（2）大于 16 的计数器与倒计时的实现

在实现 24 进制计数器的过程中，出现了一些问题：

- 初始设计的清零逻辑未完全考虑高低位组合状态，导致计数器无法准确在 23（10111）状态后复位。经过仔细分析，增加了更明确的复位逻辑条件，明确高低位的特定组合才触发清零信号。

在实现倒计时功能时，也遇到了以下问题：

- 初始时不知道该如何实现减法，做成了从 10 计到 15 后返回 10 的样子。后来修改 74LS161 的逻辑，才实现了减法倒计时的基础功能。
- 倒计时过程中，最初设计减法逻辑时，未处理好零点溢出的问题，导致计数器在减到零时无法正确跳转回初始值。通过明确定义计数值减到零后的自动置数逻辑，成功实现了循环倒计时。
- 仿真时出现由 0 开始，直接跳到 15 开始倒计时的现象。经过修改清零信号、置数信号等信号的设置，成功实现从 5 到 0 的六进制倒计时。

2、心得体会

本次实验强化了对 FPGA 芯片编程与硬件设计调试能力的提升。通过分析竞争冒险现象，我深刻体会到同步逻辑设计的重要性，以及在 FPGA 设计中的时序考虑。自主设计和扩展功能的部分进一步提升了我的创新能力与对 FPGA 系统理解的深度。